

Section 7 GenAI Task – Testing Registration

Team Number	Jonathan Mai
Backend Language	Python
Testing Framework	pytest with pytest-django

Reflective Essay

My overall experience with using ChatGPT to generate a test suite was very detailed; it produced serviceable code on the first try without needing to be corrected on test logic. The main reason I believe that to be the case is that, in general, the more context I give these generative AIs, the more correct the response it gave back to me. So in the first prompt, I gave a lot of context on what the registration does, and all the edge cases it has, so that I could ensure that it tests every edge case in the register.

The response it spits out was working perfectly fine. Since the first generated response went so well, I decided to prompt ChatGPT further by adding more routing context, as one issue with the first response was that the registration URL was hard-coded. ChatGPT responded to this issue I pointed out in the 2nd prompt by switching it to be more dynamic.

ChatGPT responded well to both follow-up prompts, the URL fix, and the GET test addition were handled cleanly with no regression in the rest of the file.

Some ethical considerations I had while I was using the AI tool are confidentiality, as this code now exists on OpenAI's servers. Another issue is that AI has no way of running these tests, especially within another one's VS Code environment, so if I were to paste this code into the project carelessly and it breaks something, the AI can't really be held responsible in that scenario.

ChatGPT Tool Chat Log.

Prompt 1:

I need you to write a backend test suite for a Django user registration feature. I am using pytest with pytest-django.

Here is the RegisterForm validation logic (marketplace/forms.py):

- Email: must end with @sfsu.edu

- Password: must be at least 8 characters; must contain at least 1 uppercase letter; must contain at least 1 digit; must contain at least 1 special character
- On save: user.email and user.sfsu_email are both set to the lowercased email; a UserProfile is also created

Please generate tests that cover:

- Successful registration with valid inputs → user is created, logged in, and redirected
- Registration with a non-@sfsu.edu email → form error, no user created
- Registration with a duplicate email (case-insensitive) → form error
- Registration with a duplicate username → form error
- Password too short (under 8 characters) → form error
- Password missing an uppercase letter → form error
- Password missing a digit → form error
- Password missing a special character → form error
- Mismatched passwords (password1 ≠ password2) → form error

GENERATED RESPONSE:

I'll give you a ready-to-drop-in pytest file. I'll keep the assertions focused on the view behavior, form errors, database changes, login session, and profile creation.

Here is a complete **pytest-django** backend test suite you can place in something like:

marketplace/tests/test_registration.py

```
import pytest
```

```
from django.contrib.auth import get_user_model
```

```
from django.urls import reverse
```

```
from marketplace.models import UserProfile
```

```
pytestmark = pytest.mark.django_db
```

```
User = get_user_model()
```

```
REGISTER_URL = "/register/"
```

```
def valid_registration_data(
    username="teststudent",
    email="teststudent@sfsu.edu",
    password="StrongPass1!",
):
    return {
        "username": username,
        "email": email,
        "password1": password,
        "password2": password,
    }
```

```
def create_existing_user(username="existinguser", email="existing@sfsu.edu"):
```

```
    """
```

Helper for duplicate username/email tests.

Adjust this if your custom user manager requires different fields.

```
    """
```

```
    user = User.objects.create_user(
        username=username,
```

```
    email=email.lower(),  
    password="ExistingPass1!",  
)
```

```
if hasattr(user, "sfsu_email"):  
    user.sfsu_email = email.lower()  
    user.save()
```

```
return user
```

```
def assert_form_has_error(response, field_name):  
    form = response.context["form"]  
    assert field_name in form.errors  
    assert len(form.errors[field_name]) > 0
```

```
def test_successful_registration_creates_user_logs_in_and_redirects(client):  
    data = valid_registration_data(  
        username="validstudent",  
        email="ValidStudent@SFSU.EDU",  
        password="StrongPass1!",  
    )
```

```
response = client.post(REGISTER_URL, data)
```

```
assert response.status_code == 302
```

```
assert response.url == reverse("marketplace_home")
```

```
assert User.objects.count() == 1
```

```
user = User.objects.get(username="validstudent")
```

```
assert user.email == "validstudent@sfsu.edu"
```

```
assert user.sfsu_email == "validstudent@sfsu.edu"
```

```
assert UserProfile.objects.filter(user=user).exists()
```

```
assert str(user.pk) == client.session.get("_auth_user_id")
```

```
def test_registration_with_non_sfsu_email_shows_form_error_and_creates_no_user(client):
```

```
    data = valid_registration_data(
```

```
        username="bademailuser",
```

```
        email="bademail@gmail.com",
```

```
        password="StrongPass1!",
```

```
    )
```

```
response = client.post(REGISTER_URL, data)
```

```
assert response.status_code == 200
```

```
assert User.objects.count() == 0
```

```
assert_form_has_error(response, "email")
```

```
def test_registration_with_duplicate_email_case_insensitive_shows_form_error(client):
```

```
    create_existing_user(
```

```
        username="originaluser",
```

```
        email="student@sfsu.edu",
```

```
    )
```

```
    data = valid_registration_data(
```

```
        username="newuser",
```

```
        email="STUDENT@SFSU.EDU",
```

```
        password="StrongPass1!",
```

```
    )
```

```
    response = client.post(REGISTER_URL, data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 1
```

```
    assert_form_has_error(response, "email")
```

```
def test_registration_with_duplicate_username_shows_form_error(client):
```

```
    create_existing_user(
        username="takenusername",
        email="taken@sfsu.edu",
    )
```

```
    data = valid_registration_data(
        username="takenusername",
        email="newstudent@sfsu.edu",
        password="StrongPass1!",
    )
```

```
    response = client.post(REGISTER_URL, data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 1
```

```
    assert_form_has_error(response, "username")
```

```
def test_registration_with_password_too_short_shows_form_error(client):
```

```
    data = valid_registration_data(
        username="shortpassuser",
```

```
    email="shortpass@sfsu.edu",  
    password="Aa1!",  
)
```

```
response = client.post(REGISTER_URL, data)
```

```
assert response.status_code == 200  
assert User.objects.count() == 0  
assert_form_has_error(response, "password1")
```

```
def test_registration_with_password_missing_uppercase_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nouppercaseuser",  
        email="nouppercase@sfsu.edu",  
        password="lowercase1!",  
    )
```

```
response = client.post(REGISTER_URL, data)
```

```
assert response.status_code == 200  
assert User.objects.count() == 0  
assert_form_has_error(response, "password1")
```



```
def test_registration_with_password_missing_digit_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nodigituser",  
        email="nodigit@sfsu.edu",  
        password="NoDigitPass!",  
    )
```

```
    response = client.post(REGISTER_URL, data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 0
```

```
    assert_form_has_error(response, "password1")
```

```
def test_registration_with_password_missing_special_character_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nospecialuser",  
        email="nospecial@sfsu.edu",  
        password="NoSpecial1",  
    )
```

```
    response = client.post(REGISTER_URL, data)
```

```
assert response.status_code == 200

assert User.objects.count() == 0

assert_form_has_error(response, "password1")
```

```
def test_registration_with_mismatched_passwords_shows_form_error(client):
```

```
    data = {

        "username": "mismatchuser",

        "email": "mismatch@sfsu.edu",

        "password1": "StrongPass1!",

        "password2": "DifferentPass1!",

    }
```

```
    response = client.post(REGISTER_URL, data)
```

```
    assert response.status_code == 200

    assert User.objects.count() == 0

    assert_form_has_error(response, "password2")
```

One possible small adjustment: if your register URL has a name, such as `name="register"`, you can replace:

```
REGISTER_URL = "/register/"
```

with:

```
REGISTER_URL = reverse("register")
```

Prompt 2:

The register url does have a name here is the registration view (register_view in marketplace/views.py):

- It accepts POST requests at /register/
- It uses a form called RegisterForm (a subclass of Django's UserCreationForm)
- On valid form submission, it saves the user, auto-logs them in, and redirects to marketplace_home
- On invalid submission, it re-renders the registration page with errors
- If the user is already authenticated, it immediately redirects to marketplace_home

Otherwise test suite looks good. Please add one more test: a GET request to /register/ by an unauthenticated user should return a 200 status and include a form in the response context.

GENERATED RESPONSE:

Got it. Replace the hardcoded URL with `reverse("register")`, and add the unauthenticated GET test.

Updated test file:

```
import pytest
```

```
from django.contrib.auth import get_user_model
```

```
from django.urls import reverse
```

```
from marketplace.forms import RegisterForm
```

```
from marketplace.models import UserProfile
```

```
pytestmark = pytest.mark.django_db
```

```
User = get_user_model()
```

```
def valid_registration_data(
```

```
username="teststudent",  
email="teststudent@sfsu.edu",  
password="StrongPass1!",  
):  
  
    return {  
        "username": username,  
        "email": email,  
        "password1": password,  
        "password2": password,  
    }
```

```
def create_existing_user(username="existinguser", email="existing@sfsu.edu"):  
  
    user = User.objects.create_user(  
        username=username,  
        email=email.lower(),  
        password="ExistingPass1!",  
    )  
  
    if hasattr(user, "sfsu_email"):  
        user.sfsu_email = email.lower()  
        user.save()  
  
    return user
```

```
def assert_form_has_error(response, field_name):
```

```
    form = response.context["form"]
```

```
    assert field_name in form.errors
```

```
    assert len(form.errors[field_name]) > 0
```

```
def test_get_register_page_unauthenticated_returns_200_and_form(client):
```

```
    response = client.get(reverse("register"))
```

```
    assert response.status_code == 200
```

```
    assert "form" in response.context
```

```
    assert isinstance(response.context["form"], RegisterForm)
```

```
def test_successful_registration_creates_user_logs_in_and_redirects(client):
```

```
    data = valid_registration_data(
```

```
        username="validstudent",
```

```
        email="ValidStudent@SFSU.EDU",
```

```
        password="StrongPass1!",
```

```
    )
```

```
    response = client.post(reverse("register"), data)
```

```
assert response.status_code == 302
```

```
assert response.url == reverse("marketplace_home")
```

```
assert User.objects.count() == 1
```

```
user = User.objects.get(username="validstudent")
```

```
assert user.email == "validstudent@sfsu.edu"
```

```
assert user.sfsu_email == "validstudent@sfsu.edu"
```

```
assert UserProfile.objects.filter(user=user).exists()
```

```
assert str(user.pk) == client.session.get("_auth_user_id")
```

```
def test_registration_with_non_sfsu_email_shows_form_error_and_creates_no_user(client):
```

```
    data = valid_registration_data(
```

```
        username="bademailuser",
```

```
        email="bademail@gmail.com",
```

```
        password="StrongPass1!",
```

```
    )
```

```
    response = client.post(reverse("register"), data)
```

```
assert response.status_code == 200

assert User.objects.count() == 0

assert_form_has_error(response, "email")
```

```
def test_registration_with_duplicate_email_case_insensitive_shows_form_error(client):
```

```
    create_existing_user(
        username="originaluser",
        email="student@sfsu.edu",
    )
```

```
    data = valid_registration_data(
        username="newuser",
        email="STUDENT@SFSU.EDU",
        password="StrongPass1!",
    )
```

```
    response = client.post(reverse("register"), data)
```

```
    assert response.status_code == 200

    assert User.objects.count() == 1

    assert_form_has_error(response, "email")
```

```
def test_registration_with_duplicate_username_shows_form_error(client):
```

```
    create_existing_user(  
        username="takenusername",  
        email="taken@sfsu.edu",  
    )
```

```
    data = valid_registration_data(  
        username="takenusername",  
        email="newstudent@sfsu.edu",  
        password="StrongPass1!",  
    )
```

```
    response = client.post(reverse("register"), data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 1
```

```
    assert_form_has_error(response, "username")
```

```
def test_registration_with_password_too_short_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="shortpassuser",  
        email="shortpass@sfsu.edu",
```



```
        password="Aa1!",  
    )
```

```
response = client.post(reverse("register"), data)
```

```
assert response.status_code == 200
```

```
assert User.objects.count() == 0
```

```
assert_form_has_error(response, "password1")
```

```
def test_registration_with_password_missing_uppercase_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nouppercaseuser",  
        email="nouppercase@sfsu.edu",  
        password="lowercase1!",  
    )
```

```
response = client.post(reverse("register"), data)
```

```
assert response.status_code == 200
```

```
assert User.objects.count() == 0
```

```
assert_form_has_error(response, "password1")
```

```
def test_registration_with_password_missing_digit_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nodigituser",  
        email="nodigit@sfsu.edu",  
        password="NoDigitPass!",  
    )
```

```
    response = client.post(reverse("register"), data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 0
```

```
    assert_form_has_error(response, "password1")
```

```
def test_registration_with_password_missing_special_character_shows_form_error(client):
```

```
    data = valid_registration_data(  
        username="nospecialuser",  
        email="nospecial@sfsu.edu",  
        password="NoSpecial1",  
    )
```

```
    response = client.post(reverse("register"), data)
```

```
    assert response.status_code == 200
```

```
assert User.objects.count() == 0
```

```
assert_form_has_error(response, "password1")
```

```
def test_registration_with_mismatched_passwords_shows_form_error(client):
```

```
    data = {
```

```
        "username": "mismatchuser",
```

```
        "email": "mismatch@sfsu.edu",
```

```
        "password1": "StrongPass1!",
```

```
        "password2": "DifferentPass1!",
```

```
    }
```

```
    response = client.post(reverse("register"), data)
```

```
    assert response.status_code == 200
```

```
    assert User.objects.count() == 0
```

```
    assert_form_has_error(response, "password2")
```

If your URL name is not exactly `"register"`, change each `reverse("register")` to your actual route name, for example `reverse("marketplace_register")`.